

WildRydes AWS Implementation

Colin MacPherson

Content

1 What is WildRydes?



2 System Architecture



3 Implementation



4 Testing



5 Costs



6 Conclusion



What is WildRydes?

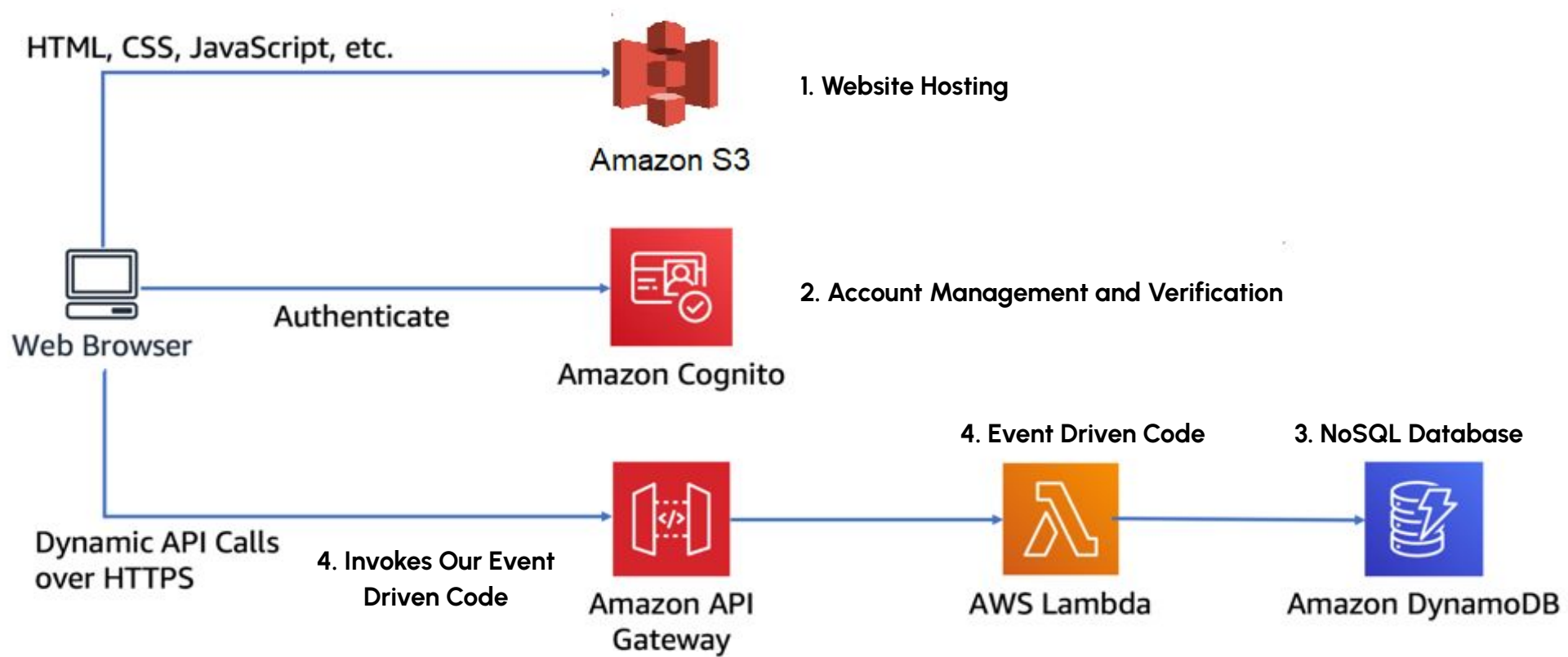


WildRydes is an innovative transportation service that provides flying unicorn rides as a means to get people to their destinations quicker and easier.

What is needed to support this service?

1. Some sort of interface for users to request rides
 - a. **In our case: a website**
2. Secure login and accounts to ensure rider safety
 - a. **In our case: an account management system**
3. A database system to ensure data security and accuracy
 - a. **In our case: a NoSQL database**
4. Automated processing so users can be matched with a flying unicorn and receive direct updates
 - a. **In our case: an event driven way to run our code and update the user**



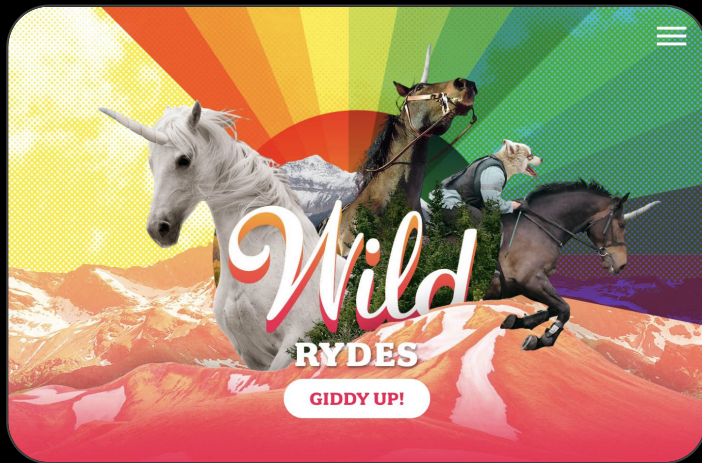


System Architecture



Website Hosting - S3

- S3 is a scalable, durable, and available object storage service that also provides static website hosting.
- S3 is used to host our WildRydes website including all the HTML, CSS, JavaScript, and image files
- This is the foundation of our architecture as it provides the user interface for which all of the other aspects are accessed and utilized.
 - On the user side, account management and requesting rides — which invokes our API, event driven code, and database — is all done through this website.



WildRydes Homepage



Website Hosting - S3

- **HTML:** The HTML in S3 acts as the framework for our different website pages. Specifically, it structures the items on the page by defining section dividers, classes, text options — headers, subtitles, regular text, and links.
- **CSS:** This is the styling for the website. It determines how these different structure items actually appear on the page. Some examples include: centering text/images, fonts, font-size, locations on the page, whether they're fixed or floating, etc.
- **JavaScript:** JS adds the interactive and dynamic elements to our website. It is responsible for the dropdown menu on the top-right and invoking our AWS services such as Cognito, API Gateway, Lambda, and DynamoDB

index.html



unicorns.html



Ex. cognito-auth.js

Redirects the users to the signin.html

Shows an error if config.js is missing

Initializes a CognitoUserPool object

Adds a sign out method to WildRydes

```

const WildRydes = {
  config: {
    AmazonCognitoIdentity: {
      region: 'us-east-1',
      userPoolId: 'us-east-1:XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX'
    },
    AmazonCognitoIdentityServiceProvider: {
      region: 'us-east-1',
      userPoolId: 'us-east-1:XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX'
    },
    AmazonCognitoIdentityProvider: {
      region: 'us-east-1',
      userPoolId: 'us-east-1:XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX'
    },
    AmazonCognitoIdentityProvider: {
      region: 'us-east-1',
      userPoolId: 'us-east-1:XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX'
    }
  },
  init: function() {
    if (!this.config) {
      console.error('config.js is missing');
      return;
    }
    const { AmazonCognitoIdentity, AmazonCognitoIdentityServiceProvider, AmazonCognitoIdentityProvider } = this.config;
    const userPool = new AmazonCognitoIdentity.CognitoUserPool({
      poolData: {
        UserPoolId: AmazonCognitoIdentityProvider.userPoolId,
        ClientId: AmazonCognitoIdentityProvider.clientId
      }
    });
    const cognitoUser = new AmazonCognitoIdentity.CognitoUser({
      username: 'user@example.com',
      password: 'password'
    }, userPool);
    cognitoUser.signOut();
  },
  signIn: function() {
    const cognitoUser = new AmazonCognitoIdentity.CognitoUser({
      username: 'user@example.com',
      password: 'password'
    }, userPool);
    cognitoUser.signIn();
  },
  signOut: function() {
    const cognitoUser = new AmazonCognitoIdentity.CognitoUser({
      username: 'user@example.com',
      password: 'password'
    }, userPool);
    cognitoUser.signOut();
  }
};

```



User Management - Cognito

- Cognito provides account creation and management, user authentication and authorization, and access control.
- For WildRydes, Cognito runs our entire user management system.
 - Cognito adds security to our website, users, and API, while also offering a streamlined account system.

Cognito Users Interface

The screenshot shows the Cognito Users Interface. At the top, there's a header with 'Users (3)' and an 'Info' link. Below this is a description: 'View, edit, and create users in your user pool. Users that are enabled and confirmed can sign in to your user pool.' There are buttons for 'Delete user' and 'Create user'. A search bar is present with the placeholder 'Search users by attribute'. Below the search bar is a table with the following columns: 'User name', 'Email address', 'Email verified', 'Confirmation status', and 'Status'. The table contains three rows of user data.

	User name	Email address	Email verified	Confirmation status	Status
☐	180173a0-e091-70ce-9...	colin1124macpherson...	Yes	Confirmed	Enabled
☐	1801b350-4061-70ac-c...	dadadlaksldl@gmail.c...	No	Confirmed	Enabled
☐	c871e370-e011-70b5-...	ddoad@gmail.com	No	Unconfirmed	Enabled

The Users page in Cognito provides information on all the users that have created accounts with WildRydes. It also includes whether or not they have been verified and the method of verification.

In the above image: We can see that the first two users have been confirmed, although one did not get verified through their email. It also shows that one user is not verified and therefore will not have access to our API and service.

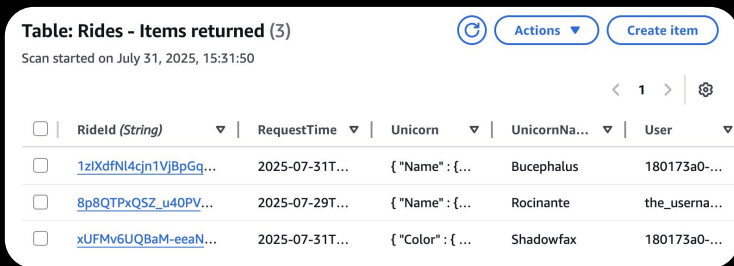


Database - DynamoDB

- DynamoDB is fully managed NoSQL database service
- For WildRydes, DynamoDB is used to store and retrieve data, specifically the data associated with each users' request for a ride.
- The following data is stored in our DynamoDB table:
 - Ride ID
 - Time of the Request
 - Unicorn Name
 - Username
- Where does this fit in the system?
 - The user interacts with the website
 - JavaScript triggers an API call*
 - Our event driven function process the request*
 - The data from the request is stored in DynamoDB

* discussed further in the next sections

DynamoDB Rides Table



	RideId (String)	RequestTime	Unicorn	UnicornNa...	User
☐	1zIXdfNl4cjp1VjBpGg...	2025-07-31T...	{ "Name": { ...	Bucephalus	180173a0-...
☐	8p8QTPxQSZ_u40PV...	2025-07-29T...	{ "Name": { ...	Rocinante	the_userna...
☐	xUFMv6UQBaM-eeaN...	2025-07-31T...	{ "Color": { ...	Shadowfax	180173a0-...

The **Rides** table stores all our data on each requested ride.

Using the data from our Rides table we can offer ride history to our users and status tracking based on the initial request time, or improve our own services and spending by completing analysis on request traffic throughout the day.



Event-Driven Functions

- Lambda

- Lambda is an event-driven service that offers serverless computing for running code.
- This is perfect for WildRydes because we need to automatically process and dispatch a unicorn whenever a user requests a ride.
- The code (partially shown on the right) completes the following actions:
 - Validates the user identity through Cognito
 - Receives the request and assigns a unicorn to the given pickup location
 - Records the ride data to DynamoDB
 - Updates the ride details to the user

Lambda Function

```
function findUnicorn(pickupLocation) {
  console.log('Finding unicorn for ', pickupLocation.Latitude, ', ', pickupLocation.Longitude);
  return fleet[Math.floor(Math.random() * fleet.length)];
}

async function recordRide(rideId, username, unicorn) {
  const params = {
    TableName: 'Rides',
    Item: marshall({
      RideId: rideId,
      User: username,
      Unicorn: unicorn,
      UnicornName: unicorn.Name,
      RequestTime: new Date().toISOString(),
    }),
  };

  await ddbClient.send(new PutItemCommand(params));
}
```



Backend Connection - API Gateway

- API Gateway is a fully managed service for all things related to creating and managing secure APIs.
- For WildRydes, API Gateway acts as the bridge between our website/users and our backend services (Lambda, DynamoDB)
- When a user request a ride, our JavaScript invokes API Gateway, routing the request to the correct Lambda function.
 - Without API Gateway we would not have access to our event-driven functions.

API Gateway Request/Response Cycle



1. User makes a request
2. Request is prepared for Lambda
3. Lambda function is triggered with the request payload
4. Lambda response is passed back to API Gateway
5. The final request is sent back to the client and the user will see it on the website

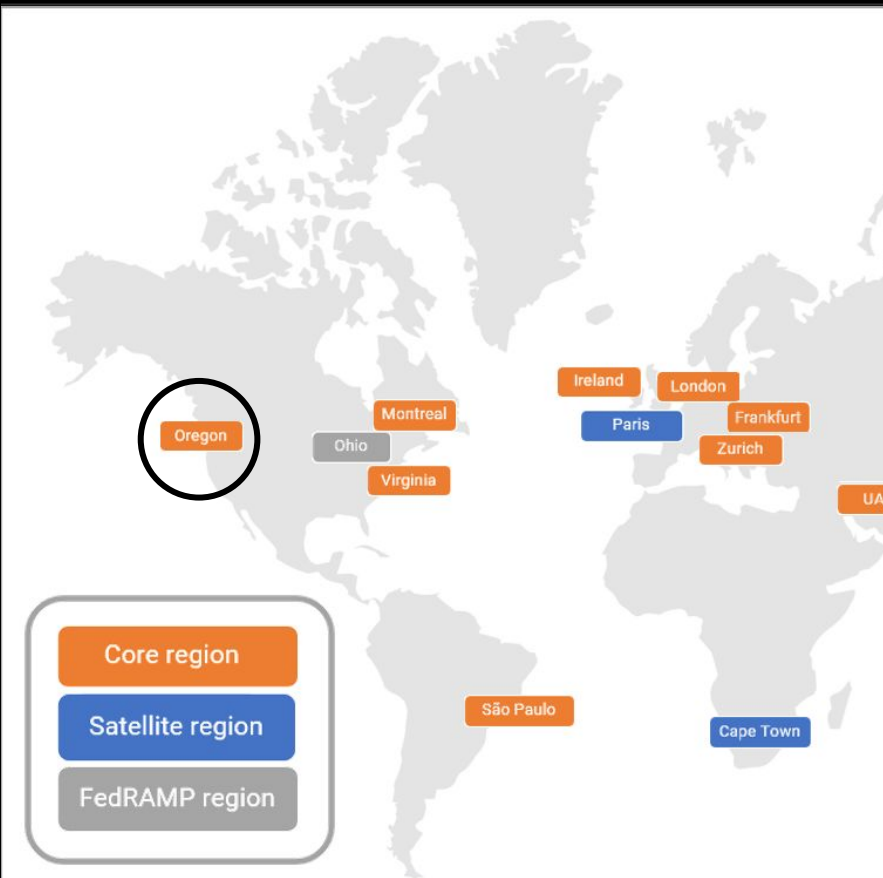
Implementation & Testing in AWS



How was the system architecture implemented in AWS and what design choices were made? How were they tested to ensure they were working properly

Region Selection — US-West-2 Oregon

Region selection is important in AWS as different regions support different services. US-West-2 (Oregon) provides access to all the aforementioned services and is located close to main area of operation (the West Coast of USA) minimizing latency to our users. Therefore, it was a straightforward choice to use as our region.



Genesys Cloud Regions



Website Hosting - S3

- Created two buckets in US-West-2
 - The first bucket stores our website's files
 - The second bucket stores the log entries showing who has accessed the website
- Added a bucket policy to allow public reads
 - This policy is necessary to allow anonymous user to access the website (our users)
- Enabled static website hosting for our S3 bucket to allow users to access our bucket objects directly via a public HTTP endpoint
- Testing:** To test to see if our website is properly accessible we visited the HTTP endpoint and confirmed that the website was properly working

WildRydes Homepage



Bucket Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::wildrydes-colin-macpherson/*"
    }
  ]
}
```

Static website hosting

Use this bucket to host a website or redirect requests. [Learn more](#)

Edit

We recommend using AWS Amplify Hosting for static website hosting
Deploy a fast, secure, and reliable website quickly with AWS Amplify Hosting. [Learn more about Amplify Hosting](#) or [View your existing Amplify apps](#)

Create Amplify app

S3 static website hosting
Enabled

Hosting type
Bucket hosting

Bucket website endpoint

When you configure your bucket as a static website, the website is available at the AWS Region-specific website endpoint of the bucket. [Learn more](#)
<http://wildrydes-colin-macpherson.s3-website-us-west-2.amazonaws.com>

Static Website Hosting

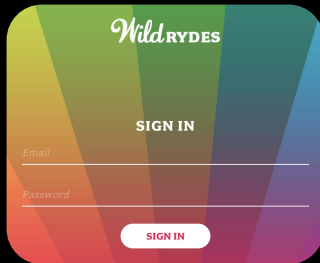
<http://wildrydes-colin-macpherson.s3-website-us-west-2.amazonaws.com/index.html>



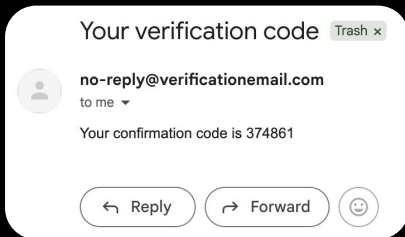
User Management - Cognito

- Created a Cognito User Pool to have sign-up and sign-in functionality on the website
 - Configured to require email authentication to improve security, communication to users, data consistency, and password recovery
 - We do not require MFA (Multi-Factor Authentication) to improve usability
- This Cognito User Pool is connected to our website via the config.js file in our S3 bucket
- Testing:** To test this system, we used the website to create an account, verified it through email, and signed in to see the /ride section of the website.

WildRydes Sign-In



Email Verification



```
window._config = {
  cognito: {
    userPoolId: 'us-west-2_QwjWUohrY', // e.g. us-east-2_uXboG5pAb
    userPoolClientId: '53qf5k90deghegtsakvhh2k6ak', // e.g. 25ddkmj4v6hfsfvrhpf17n4hv
    region: 'us-west-2' // e.g. us-west-2
  },
  api: {
    invokeUrl: 'https://9dpurl7cbb.execute-api.us-west-2.amazonaws.com/Prod' // e.g. ht
  }
};
```

config.js file



Database - DynamoDB

- Created a NoSQL database with RideID (*string*) as a partition key
- Default Settings
 - On-demand capacity to ensure it is scalable through high traffic periods
 - AWS owned key for free, simple, database security
- Testing:** Once Lambda and the API is set up and rides are requested, we can check to ensure the database has entries.

Table Information

General information [Info](#)

[Get live item count](#)

Partition key RideId (String)	Sort key -	Capacity mode On-demand	Table status Active
Alarms No active alarms	Point-in-time recovery (PITR) Info Off	Item count 3	Table size 489 bytes
Average item size 163 bytes	Resource-based policy Info Not active		

Amazon Resource Name (ARN)
 arn:aws:dynamodb:us-west-2:988448834050:table/Rides

Table: Rides - Items returned (3)

Scan started on August 02, 2025, 14:49:07

	RideId (String)	RequestTime	Unicorn	UnicornNa...	User
☐	1zIXdfNI4cjn1VjBpGg...	2025-07-31T...	{"Name": {...	Bucephalus	180173a0-...
☐	8p8QTPxQSZ_u40PV...	2025-07-29T...	{"Name": {...	Rocinante	the_userna...
☐	xUFMv6UQBaM-eeaN...	2025-07-31T...	{"Name": {...	Shadowfax	180173a0-...

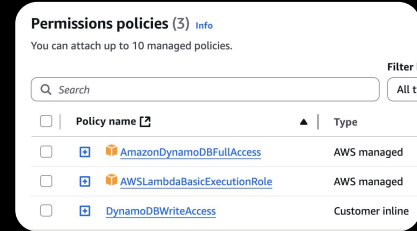


Event Driven Functions

- Lambda

- Created an **IAM role** to allow our Lambda function to write CloudWatch logs and write to our DynamoDB database. The role has the following policies:
 - AWSLambdaBasicExecutionRole (*managed*)
 - AmazonDynamoDBFullAccess (*managed*)
 - ddb:PutItem (*inline*)
- Created the Lambda function to process API requests from the website
- Testing:** To test the function we ran a test event in the Lambda console and confirmed that it was working correctly.

IAM Permission Policies

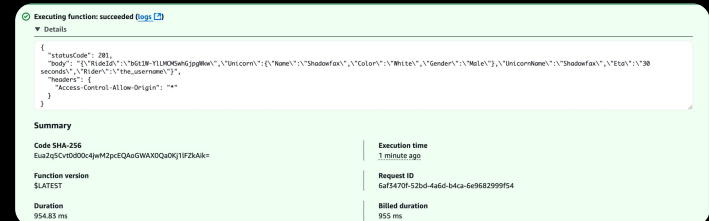


Test Event

```

1 * @
2 *path: "/ride",
3 *httpMethod: "POST",
4 *
5 *headers: {
6 *   "Accept": "*/**",
7 *   "Authorization": "eyJrdWQ0LjI1ZmRlZWZs",
8 *   "Content-Type": "application/json; charset=UTF-8"
9 * },
10 *queryStringParameters: null,
11 *pathParameters: null,
12 *requestContext: {
13 *   "authorizer": {
14 *     "claims": {
15 *       "cognito:username": "the_username"
16 *     }
17 *   }
18 * },
19 *body: "{\"PickupLocation\":{\"Latitude\":47.617475583563,\"Longitude\":-122.28837066650185}}"
```

Test Confirmation





Backend Connection

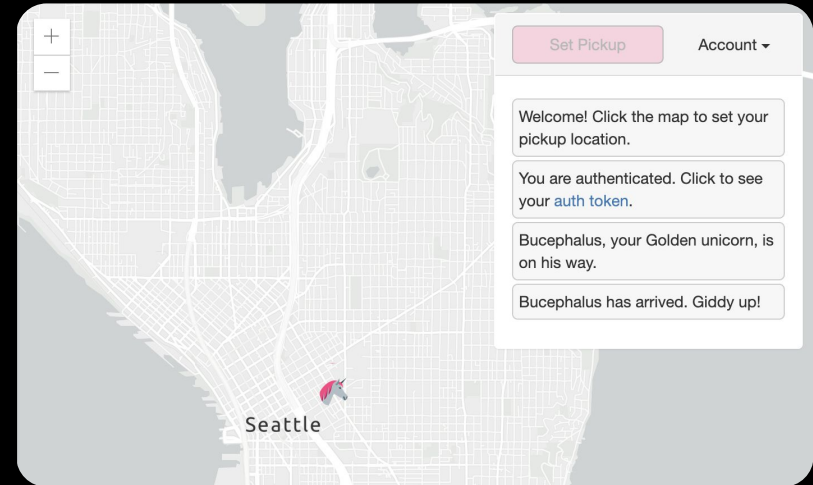
- API Gateway

- Created a REST API with endpoint type set to Edge Optimized
 - Using edge optimized endpoints means there is built-in global distribution and lower latency for users around the world
- Created a Cognito User Pool Authorizer for the API to authenticate the API calls
- Created a POST method to create a new resource, in this case, submit a unicorn ride request
- Deploy the API
- Update the config.js invokeURL field with invoke URL from the API
 - This ensures the website frontend is connected to our API so it is actually called when a user requests a ride

API Information

WildRydes	API for WildRydes	9dpur7cbb	REST	Edge-optimized	2025-07-31
-----------	-------------------	-----------	------	----------------	------------

Testing: The full website is up and running so all that is left is to confirm that you can request a flying unicorn!

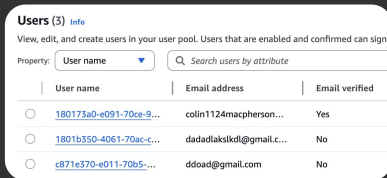


Testing Overview:



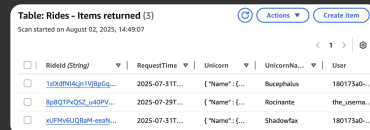
Static Website S3

Checked to ensure all pages are accessible and formatting is correct.



User Management Cognito

Checked to ensure users can create an account, verify their account, and the account shows up in AWS Cognito.



Database Entry DynamoDB

Checked to ensure ride data is correctly stored within our DynamoDB NoSQL database.

*Cannot be tested until the API is configured.



Event Driven Function Lambda

Checked to ensure our functions to process ride requests returns the correct response.

Expected Status Code: 201

Testing the **REST API** is equivalent to testing the production ready form of the website. Requesting a unicorn should properly insert the data into our database and update the user on their ride information.

Assumptions Used to Forecast AWS Usage Costs

50 million

users at steady state

10 rides

per month on average for each user

Monthly Costs Broken Down By AWS Service

S3
~\$3,500



Estimating requests from S3 standard based on the monthly active user and average usage estimates

Cognito
~\$690,000



Using the Cognito Lite tier, an estimate of 50 million monthly users yields a cost around \$690,000. This is easily the most expensive service

DynamoDB
~\$363



Using the Standard DDB table with 161.2 bytes as the average file size, the estimating 500,000,000 writes and equivalent number of reads (for completeness) are not very costly

Lambda
~\$100



Without the free tier and using the x86 architecture, the 500,000,000 function requests is fairly cheap

API Gateway
~\$175



Estimating for 50 million REST API requests, the cost is relatively low

Total Annual Costs:
8,343,363.24

Despite the high costs from AWS services, the ~500 million rides will easily produce a profit, even if offered at an average price of \$1 per ride.

Thank you

What's next?

Cost Reduction and Service
Expansion — Unicorn Delivery
Service

Colin MacPherson